Low-Level Design (LLD)

RAG-Based Customer Support Assistant

1. Module-Level Design

🔹 1. Document Processing Module

- **File:** ingestion.py
- Loads PDF using PyPDFLoader
- Extracts raw text

🔹 2. Chunking Module

- Custom **Q&A-based splitting**
- Splits document by "Q:" to preserve question-answer pairs
- Output: list of Q&A chunks

🔹 3. Embedding Module

- Uses HuggingFaceEmbeddings (all-MiniLM-L6-v2)
- Converts chunks → vector representations

🔹 4. Vector Storage Module

- Uses **ChromaDB**
- Stores embeddings with persistence (vectorstore/)

🔹 5. Retrieval Module

- **File:** retriever.py
- Uses Chroma.as_retriever()
- Retrieves top-k relevant chunks

🔹 6. Query Processing Module (RAG)

- **File:** rag.py
- Functions:

    - retrieve_context(query)
    - generate_answer(query, context)
- Applies prompt constraints to ensure grounded responses

🔹 7. Graph Execution Module

- **File:** graph.py
- Built using LangGraph
- Nodes:
    - process_node
    - decision_node
    - output_node
    - hitl_node

🔹 8. HITL Module

- **File:** hitl.py
- Returns fallback message when escalation is triggered

🔹 9. API Module

- **File:** main.py
- FastAPI endpoint:
    - /query?query=...
- Handles user input and returns response

2. Data Structures

🔹 Document Representation

Document:

```
{
    page_content: str,
    metadata: dict
}
```

- **Chunk Format**

Chunk:

"Q: <question> A: <answer>"

- **Embedding Structure**

Embedding:

List[float] # vector representation

- **Query-Response Schema**

Request:

```
{
    "query": str
}
```

Response:

```
{
    "answer": str
}
```

- **Graph State Object**

State:

```
{
    "query": str,
    "context": str,
    "answer": str
}
```

3. Workflow Design (LangGraph)

- **Nodes**

1. Process Node

- Retrieves context
- Generates answer

state → retrieve_context → generate_answer → update state

2. Decision Node

- Determines next step
- Conditions:
    - No context → HITL
    - "I don't know" → HITL
    - Else → Output

3. Output Node

- Returns final answer

4. HITL Node

- Returns escalation response

🔹 Graph Flow

```
Input → Process → Decision
            ├── Output
            └── HITL
```

4. Conditional Routing Logic

🔹 Answer Generation Criteria

- Context must not be empty
- Answer must come from context

🔹 Escalation Criteria

- No relevant chunks retrieved

- Context missing answer
- Model returns "I don’t know"

🔹 **Logic Implementation**

```
if not context:
    route = "hitl"
elif "i don't know" in answer:
    route = "hitl"
else:
    route = "output"
```

5. HITL Design

🔹 **Trigger Conditions**

- Insufficient context
- Unknown query

🔹 **Behavior**

- Returns fallback message:

"Escalated to human agent. Please wait for support."

🔹 **Integration**

- Routed via LangGraph decision node

6. API / Interface Design

🔹 **Endpoint**

POST /query?query=<user_query>

- **Input**

```
{
  "query": "string"
}
```

- **Output**

```
{
  "response": "string"
}
```

- **Interaction Flow**
  1. User sends query
  2. API calls LangGraph
  3. Graph processes query
  4. Response returned

7. Error Handling

- **Missing Data**
  - If no context → fallback to HITL

- **No Relevant Chunks**
  - Retriever returns empty → HITL

- **LLM Failure**
  - Return fallback message

- **Invalid Input**
  - Handle empty or malformed queries

8. Additional Design Considerations

- **Prompt Design**
  - Strict grounding
  - Prevent hallucination
  - Ensure complete answers

- **Retrieval Tuning**
  - k value adjusted for precision
  - Q&A chunking improves accuracy

- **Performance**
  - Local vector DB
  - Fast Groq inference